

Exact acceleration of the supplied unknot recognizer

Modular frontiers, compressed connected sums, and sparse cancellation

Implementation and mathematical report for `Knots.zip`

18 September 2026 — Version 0.2.0

Abstract

We replace the supplied `fast/` implementation by a compatible, standard-library Python recognizer with three substantive improvements: an exact one-sided modular Jones obstruction that aggregates partial smoothing states; a validated decomposition along visible connected-sum circles, with tensor products of reduced Khovanov homology kept compressed; and a faster Bar–Natan scanning backend using characteristic-two algebra, bounded memoization, and fill-in-aware cancellation. The original implementation is retained unchanged for comparisons.

On the same machine, cold-process median recognition times decrease from 35.09 ms to 0.834 ms for the Conway fixture, and from 1.372 s to 1.455 ms for two connected Conway copies. The original raw scanner does not complete the supplied 36-crossing stress case within our 30-second budget; the improved raw scanner completes in 1.487 s. Some already easy inputs become slower; all measured comparisons are reported. Independent small-cube and state-sum tests, differential checks, evidence replay, and a large-diagram Reidemeister invariance check support the implementation.

No global $n^{O(\log n)}$ running-time theorem is claimed. The general worst-case bound remains exponential. There is, however, a provable exponential improvement on a fixed-block connected-sum family, and a quasipolynomial bound on the restricted class whose extracted blocks have $O(\log^2 n)$ crossings. We explain precisely why small scan width alone does not control surviving Khovanov multiplicities, and distinguish the supplied Lackenby hierarchy results from a complete executable realization of the announced quasipolynomial algorithm.

Contents

1	Scope, deliverables, and the baseline	3
2	The quasipolynomial target and the supplied sources	3
2.1	What the talk asserts	3
2.2	What the 2026 preprint contributes	4
2.3	The exact accounting that would suffice	4
3	An exact modular Jones obstruction	5
3.1	Normalization and the one-sided decision rule	5
3.2	Frontier aggregation instead of the full state cube	5
3.3	Resource and width bounds	6
4	Visible sums and compressed homology	7
4.1	Detecting and checking a separating circle	7
4.2	Reduced ranks and the complete detector	7
4.3	Avoiding a materialized tensor product	8
5	A faster exact scanning backend	9
5.1	The local algebra	9
5.2	Separating topology from decorations	9
5.3	Cancellation and fill-in control	10
5.4	Faster crossing orders	10
6	Preprocessing, validation, and evidence	11
6.1	A linear descending test	11
6.2	An orientation fix needed for safe normalization	11
6.3	Limits are not mathematical answers	11
6.4	Replayable evidence and its trust boundary	11
7	Complexity: achieved improvements and remaining barriers	12
7.1	General worst case	12
7.2	Why width alone does not bound a materialized Khovanov complex	12
7.3	An exponential improvement for the actual recognition pipeline	13
8	Reproducible measurements	14
8.1	Protocol and environment	14
8.2	Default recognition	14
8.3	Raw scanning, separated from preprocessing	15
8.4	Compressed rank computations	15
8.5	Ordering and pivot ablations	16
9	Validation actually performed	16
10	Using and extending the implementation	17
11	Conclusion	19

1 Scope, deliverables, and the baseline

The input is a *classical one-component knot diagram* with n crossings. The implementation accepts planar-diagram (PD) codes, braid closures, and grid diagrams. A PD row (a, b, c, d) gives the four ports in cyclic order; the under-strand joins ports 0 and 2, and the over-strand joins ports 1 and 3. Each edge label must occur exactly twice. The empty PD denotes one crossing-free circle, not the empty link. The validator checks strand connectivity and that the specified rotation system is spherical. Links and virtual diagrams are rejected. The crossing number n throughout this report is the number of crossings in the *given diagram*, not the minimum crossing number of its knot type.

The supplied baseline first performs crossing-decreasing Reidemeister-I/II moves, then a descending-diagram test, then computes the full Alexander polynomial. If none decides the answer, it computes unreduced Khovanov homology over $\mathbb{F}_2$ by Bar-Natan scanning [4, 5]. The total rank divided by two is used as reduced rank. Rank one detects the unknot, as explained in Section 4.2. Resource exhaustion gives UNKNOWN, not a knot verdict.

The new default pipeline is

$$\begin{aligned} & \text{R1/R2} \longrightarrow \text{descending} \longrightarrow \text{visible sums} \\ & \longrightarrow \text{bounded Jones screen} \longrightarrow \Delta_K \longrightarrow \text{improved scan.} \end{aligned}$$

For a detected sum, the pipeline is applied to its factors, and stops as soon as a nontrivial factor is certified. The decomposition itself is performed iteratively; there is no recursion through an arbitrarily deep decomposition tree.

The package contains executable code, not calls to hypothetical hierarchy oracles. The implementation directory is `fast/fastunknot`; the unchanged supplied Python sources are in `baseline/fastunknot`. New modules are `jones.py`, `factor.py`, and `verify.py`. The existing scanning, preprocessing, Alexander, and command-line modules are modified. The tests, benchmark scripts, original stress inputs, raw measurement records, this source, and its compiled PDF are included. No external knot library, CAS, native compiler, or network connection is needed to run the recognizer.

Claim	Status in this deliverable
Exact complete mathematical decision algorithm	Yes, with unbounded resources and the standard Khovanov theorems.
Faster practical implementation	Measured on the supplied examples and structured families; regressions are included.
Exponential gain on a family	Yes: visible connected sums of a fixed nontrivial Alexander-trivial knot.
Global quasipolynomial bound	Not established and not advertised by the API.
Implementation of Lackenby’s hierarchy algorithm	No; its relevant ingredients and remaining implementation obligations are analyzed.
Formal verification of the implementation	No. Tests and replayable evidence are provided, with a stated trust boundary.

2 The quasipolynomial target and the supplied sources

2.1 What the talk asserts

The supplied talk is dated February 2021, although the accompanying preprint is from July 2026. Physical slide 14 states the quasipolynomial recognition claim. Slides 66–74 discuss a

bound of the form Lg^L , starting with linear L and exponential g . The advertised improvements use surfaces of controlled complexity, efficient hierarchy encodings, multisurfaces, and Heegaard splittings with Cheeger regions. Slide 74 gives the intended logarithmic hierarchy depth; the final slide, physical page 109, is the full control-flow diagram [2]. These are substantially different objects and operations from Khovanov scanning.

2.2 What the 2026 preprint contributes

The supplied source [3] is an important constructive foundation, not merely a statement that an algorithm exists. Its Section 9 analyzes the number of passes through steps of the hierarchy algorithm using the maximal hierarchy length L and maximal surface pattern-complexity g . Proposition 9.1 gives

$$N_{\text{steps}} \leq L(g+1)^L. \quad (1)$$

That section explicitly distinguishes the iteration count from a complete running-time analysis of all subroutines.

It would be inaccurate to stop reading there and assert that the paper has no bounds on hierarchy size or no compressed cutting procedure. Section 10 refines the construction using interior parallelity bundles and obtains a *linear* hierarchy-length bound in the initial q -complexity. Section 11 controls uniform handle types. Proposition 14.2 gives a polynomial-time construction of an admissible handle structure after the relevant cut-and-simplify operations, as a function of the number of handles and the logarithm of the surface's extended weight, under its stated hypotheses, including an essential boundary pattern. Theorem 14.4 supplies compressed cutting data and descriptions of the parallelity bundle. These results must be credited in any account of the available machinery.

Their hypotheses, encoding conventions, and branch invariants still have to be implemented and connected to the multisurface/Cheeger-region speedups in the talk. In particular, a linear bound on hierarchy length cannot simply be inserted into (1) and called quasipolynomial. Nor can an encoded surface of polynomial bit length safely be expanded into exponentially many normal discs. This report does not dispute the announced target; it does not supply that complete realization.

2.3 The exact accounting that would suffice

Lemma 2.1 (Conditional complexity calculation). *Suppose an implemented hierarchy algorithm obeys (1), has $L \leq b \log(n+2)$ and $g \leq (n+2)^a$, and every operation, including all data manipulation, costs at most $(n+2)^c$ bit operations. Here a, b, c are constants independent of the input. Its total running time is $n^{O(\log n)}$.*

Proof. The logarithm of the product of the step count and step cost is at most

$$\log L + L \log(g+1) + c \log(n+2) = O(\log^2(n+2)).$$

Exponentiation gives the result. All encoded integer sizes and expanded output sizes must be included in the per-step hypothesis. $\square$

For completeness, the base- $(g+1)$ potential underlying (1) is

$$\Phi = \sum_{i=1}^L c_i (g+1)^{L-i}, \quad 0 \leq c_i \leq g.$$

Decreasing digit c_j by one while replacing all following digits by arbitrary values at most g decreases Φ by at least

$$(g+1)^{L-j} - g \sum_{t=0}^{L-j-1} (g+1)^t = 1.$$

This controls the number of simplification cycles. It does not control the cost of finding the next surface or of representing the resulting manifold.

A faithful implementation project therefore needs concrete, tested operations on uniform handle structures and boundary patterns; compressed normal-surface and parallelity-bundle data; compression and weak-reduction certificates; and a verified implementation of the logarithmic-depth strategy. None is replaced here by enumeration hidden behind a promising function name. The improvements below instead work directly with the supplied executable algorithm.

3 An exact modular Jones obstruction

3.1 Normalization and the one-sided decision rule

Put $\delta = -A^2 - A^{-2}$. For a nonempty PD, let $s \in \{0, 1\}^n$ select a smoothing at each crossing, with smoothing 0 pairing $(a, b), (c, d)$ and smoothing 1 pairing $(a, d), (b, c)$. Write $c(s)$ for the number of closed circles. We use the unnormalized bracket

$$B_D(A) = \sum_s A^{n-2|s|} \delta^{c(s)}. \quad (2)$$

Thus a crossing-free circle has bracket δ . With the oriented crossing-sign convention used consistently by the code, writhe normalization gives

$$D \text{ represents the unknot} \implies B_D(A) = \delta(-A^3)^{w(D)}. \quad (3)$$

This is the Kauffman state model with a loop-normalization factor [8]. The usual convention with bracket one on a circle differs by that factor.

Choose an integer modulus $m \geq 2$ and an integer a coprime to m . Evaluation at $A = a$ defines a homomorphism from $\mathbb{Z}[A, A^{-1}]$ to $\mathbb{Z}/m\mathbb{Z}$. In particular, negative writhe causes no problem: $-a^3$ is a unit. The default uses $a = 2$ and $m = 1\,000\,000\,007$.

Proposition 3.1 (Sound obstruction). *If the two sides of (3), evaluated modulo m , differ, then D represents a nontrivial knot. Equality is inconclusive.*

Proof. An equality in the Laurent polynomial ring remains an equality under every ring homomorphism. Therefore a modular inequality contradicts the necessary unknot identity. The converse does not follow: evaluation can lose information, even when two Laurent polynomials differ. $\square$

There is no probabilistic acceptance, no primality assumption on m , and no modular division by δ . Even $\delta = 0$ is safe, though the test then has no power on a nonempty knot diagram. The implementation never turns agreement of one evaluation, or even agreement of the full Jones polynomial, into an unknot verdict.

3.2 Frontier aggregation instead of the full state cube

After some crossings have been processed, the smoothing in that region consists of closed circles and paths joining the current boundary edge midpoints. Absorb the circles into the coefficient.

The remaining combinatorial state is a perfect matching π of the boundary labels. Maintain a dictionary

$$F_i : \{\text{boundary matchings}\} \longrightarrow \mathbb{Z}/m\mathbb{Z}.$$

Its value at π is the sum of the weights of *all* partial smoothing choices inducing π .

To add a crossing, try its two smoothings for each dictionary entry. A small disjoint-set forest joins old matching arcs and new smoothing arcs. Every connected component has either two new boundary ends or no boundary ends. The former gives a new matching arc; the latter contributes a factor δ . Multiply by a or a^{-1} , and add the result to the new matching's residue. Entries that cancel to zero are deleted immediately. No cobordism compositions or Khovanov basis vectors are used by this routine.

Proposition 3.2 (Frontier invariant). *After processing any prefix of the chosen crossing order, $F_i(\pi)$ is exactly the sum of the corresponding partial state weights. At the final stage, $F_n(\emptyset) = B_D(a) \pmod{m}$.*

Proof. Before any crossing, the empty partial state has weight one. Every partial state at stage $i + 1$ is uniquely obtained by extending a state at stage i with one of two smoothings. The transition multiplies by exactly its smoothing coefficient and the factors for newly closed circles. Summing states with identical outgoing matchings is distributivity in the coefficient ring. Induction proves the first claim. There are no boundary ends after the last crossing, giving the second. The empty-PD single-circle case is handled separately with value δ . $\square$

3.3 Resource and width bounds

Let w be the largest number of frontier ends. A safe state-count bound is

$$S(w) \leq (w - 1)!! = 2^{O(w \log(w+2))} \quad (w > 0 \text{ even}), \quad (4)$$

with $S(0) = 1$. It is tempting to replace this by a Catalan number, but that requires an appropriate common cyclic ordering on a disk boundary. An arbitrary partial crossing scan can have several boundary components. We do not use an unproved disk-boundary assumption in the complexity claim.

A deliberately conservative upper bound for one transition in the current Python representation is $O((w + 1)^2)$ elementary operations, including the unranked disjoint-set operations and canonical sorting. For fixed-size modular arithmetic, the uncapped evaluation therefore takes

$$O(nS(w)(w + 1)^2)$$

operations, in addition to finding an order, and stores $O(S(w)(w + 1))$ label and residue words. For variable m , include the cost of arithmetic on $O(\log m)$ -bit integers and the initial modular inverses.

The *default* screen stops as soon as the partially built dictionary exceeds 4096 entries. At most 4097 entries are present at the triggering instant, though the previous dictionary also remains live. The cap can trigger before later cancellations would shrink that dictionary; this sacrifices an opportunity to screen, not correctness. Hitting this local state ceiling skips the screen and continues to Alexander/Khovanov. Exhausting the global time budget gives UNKNOWN. Thus an unlucky Jones evaluation cannot become an unbounded mandatory exponential prepass in the default recognizer.

4 Visible sums and compressed homology

4.1 Detecting and checking a separating circle

For the connected plane projection graph G , compute the incident faces of each edge. Two edges with the same two *distinct* incident faces are parallel edges in the plane dual. Their union in the dual supplies a two-edge cycle and hence a candidate circle meeting G in exactly two edge interiors.

The implementation groups edges by their unordered pair of incident face IDs. It tests consecutive entries in each group; any two distinct parallel dual edges already give a dual 2-cycle, so their order in the group need not be geometric. For a proposed pair $\{e, f\}$, a graph traversal after deleting those two edges finds a proposed vertex side V_1 . Before accepting the cut, the program checks that V_1 is nonempty and proper, that its entire edge boundary is precisely $\{e, f\}$, that the two shared dual endpoints are distinct, and that both sides are connected. It closes each side by identifying the two cut labels, and checks both resulting PDs for spherical rotation systems and exactly one strand component.

Lemma 4.1 (Meaning of an accepted cut). *An accepted cut decomposes the represented knot as the connected sum of the two returned knot diagrams, with no crossing added or removed.*

Proof. The two face arcs joining the cut-edge interiors form an embedded dual 2-cycle. It is a Jordan curve in the projection sphere and avoids all crossings. The checked edge boundary means it meets the projection exactly twice. A narrow three-dimensional thickening gives the usual separating sphere meeting the knot in two points. Each side is a one-strand tangle. Capping the two endpoints along the separating circle is exactly the label identification performed by the code. Its inverse operation is connected sum. The crossings and their cyclic port data are inherited unchanged on each side. $\square$

All accepted cuts are recorded in a tree with parent and child node IDs, the two cut-edge labels, and the crossing indices on one side. The verifier rebuilds and checks each cut, then validates the leaf list. An iterative worklist avoids the Python recursion limit. A conservative bound for the current repeated-search implementation is $O(n^3)$ word operations: $O(n)$ candidates, each checked in $O(n)$ time, at each of at most $n - 1$ splits. Sorting and integer-label bit costs add polynomial overhead. This is not an optimized general graph-bond routine.

This decomposition is *not topological prime factorization*. A composite knot can have a projection with no visible two-edge sum cut. No conclusion about primality is made when the procedure returns one factor.

4.2 Reduced ranks and the complete detector

Write

$$r(K) = \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2), \quad P_D(t) = \sum_h \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}^h(D; \mathbb{F}_2) t^h.$$

Here the code records the *raw cube degree* h , the number of 1-smoothings, not the usual writhe-shifted homological degree, and does not compute quantum gradings. Raw degrees are additive under cuts that preserve the set of crossings.

Over $\mathbb{F}_2$, unreduced homology consists of two copies of reduced homology with quantum shifts and no homological shift [7, Corollary 3.2.C]. Consequently every unreduced homological-degree rank used here is even. An algebraic explanation is useful. On a resolution with circle

variables $x_1, \dots, x_c$, let X multiply by the basepoint-circle variable and let $\nu = \sum_i \partial/\partial x_i$. In characteristic two,

$$X^2 = \nu^2 = 0, \quad X\nu + \nu X = \text{id}.$$

The maps commute with the Khovanov differential: this is checked on the multiplication and comultiplication of $\mathbb{F}_2[x]/(x^2)$. Thus $X\nu$ is a chain projection onto $\text{im } X$, the reduced subcomplex up to quantum shift, and the other summand is isomorphic to it via X and ν . Forgetting quantum degree gives the asserted doubling.

The complete decision rule is

$$K \text{ is the unknot} \iff r(K) = 1. \quad (5)$$

Universal coefficients relate the field to the standard detection theorem:

$$\dim_{\mathbb{Q}} \widetilde{\text{Kh}}(K; \mathbb{Q}) \leq r(K).$$

Reduced rational homology is nonzero, since its graded Euler characteristic is the normalized Jones polynomial and that polynomial has value one at one. Hence $r(K) = 1$ implies reduced rational rank one, to which Kronheimer–Mrowka’s theorem applies [6]. Conversely, the unknot has reduced rank one over $\mathbb{F}_2$. We do not assume that torsion-free rational computations can always replace the $\mathbb{F}_2$ calculation.

Proposition 4.2 (Connected-sum product). *For the two diagrams returned by an accepted cut,*

$$P_D(t) = P_{D_1}(t)P_{D_2}(t), \quad r(K_1 \# K_2) = r(K_1)r(K_2).$$

Proof. Place each basepoint on the arc being joined. A pair of smoothing states on the two sides determines exactly one state of the sum. The two distinguished circles are joined to one distinguished circle; in the reduced complex each carries the fixed basepoint label x . All remaining circle labels are independent. This identifies the reduced cube for the sum with the tensor product of the reduced cubes, with additive cube degree. Saddle maps in either block agree under this identification. The tensor-product differential has no sign issue over $\mathbb{F}_2$. Künneth over a field yields the homological-degree convolution and then the rank product. $\square$

Every factor has positive reduced rank. Combining the product with (5), the sum is the unknot precisely when all factors are. This justifies both the early nontrivial-factor exit and the all-factors-unknot branch. No unimplemented prime-decomposition oracle is needed for this rule.

4.3 Avoiding a materialized tensor product

The raw scanner still produces one object per surviving unreduced generator. The new `factored_khovanov_rank` instead computes each factor’s reduced homological polynomial and multiplies these polynomials using integer coefficient convolutions. The output is a map from degree to a binary integer rank, not a list of generators. Identical *normalized PD tuples* can reuse a factor computation. This is exact tuple memoization, not isotopy testing and not a complete canonical representation of knots.

The sum of factor crossing counts is n , and all output degrees are between zero and n . Coefficients have $O(n)$ bits under the elementary exponential bound on cube size. Ordinary coefficient convolutions therefore have polynomial bit complexity. Even without the memoization, repeated bounded-size factors cost only polynomial time to process. The extra unreduced factor is inserted once:

$$\dim_{\mathbb{F}_2} \text{Kh}(K_1 \# \dots \# K_k; \mathbb{F}_2) = 2 \prod_{i=1}^k r(K_i),$$

not $\prod_i 2r(K_i)$.

5 A faster exact scanning backend

5.1 The local algebra

The baseline represents a boundary object by a matching. Morphisms are $\mathbb{F}_2$ -linear combinations of dotted cobordisms in a normal form indexed by dot subsets on circles obtained by closing source against target. Composition uses the Frobenius algebra $A = \mathbb{F}_2[x]/(x^2)$, with

$$\frac{m}{\quad} \left| \begin{array}{cccc} 1 \otimes 1 & 1 \otimes x & x \otimes 1 & x \otimes x \\ 1 & x & x & 0 \end{array} \right., \quad \Delta(1) = 1 \otimes x + x \otimes 1, \quad \Delta(x) = x \otimes x.$$

An undotted sphere is zero, a once-dotted sphere is one, and an extra dot gives zero. A handle gives $m\Delta = 0$ in characteristic two. Delooping a circle replaces it by two summands. These are the same mathematical operations as in the supplied backend, not a different homology theory [5, 4].

For an object consisting of r arcs, the endomorphism algebra in this normal form is

$$R_r = \mathbb{F}_2[x_1, \dots, x_r]/(x_1^2, \dots, x_r^2). \quad (6)$$

A squarefree monomial is stored as a set of dotted components, and a polynomial as the set of monomials with coefficient one. Addition is symmetric difference.

Lemma 5.1 (Every unit is self-inverse). *An element of R_r is a unit exactly when its constant coefficient is one. Every such unit satisfies $u^{-1} = u$.*

Proof. Write $u = \epsilon + \eta$, where η has zero constant coefficient. In characteristic two, commutativity gives $\eta^2 = \sum_M M^2 = 0$, since every nonconstant squarefree monomial has a squared variable. If $\epsilon = 1$, $u^2 = 1$; if $\epsilon = 0$, $u^2 = 0$, so u cannot be invertible. $\square$

The baseline builds an inverse by a finite nilpotent series using general surface composition. The replacement needs only a copy of the input monomial set. It also recognizes identity compositions, and multiplies endomorphism monomials directly: intersecting dot sets give zero; otherwise their union is the product. General source/intermediate/target combinations still use the full surface calculation.

5.2 Separating topology from decorations

The topological part of composing $a \rightarrow b \rightarrow c$ depends only on the three matchings. It determines the connected surface components, their Euler characteristics, and their outgoing boundary circles. Dot decorations are then distributed over this fixed plan. Previously this topology was repeatedly reconstructed for different decorations.

The implementation now caches topology plans, complete decorated compositions, and the local crossing-tensor/delooping entries. Cached decorated values are immutable; public morphism results are fresh mutable sets, so subsequent symmetric differences cannot corrupt cached results. The entry ceilings are 4096 each for circle decompositions, gluing records, and composition plans; 16384 for decorated compositions; and 8192 for crossing entries. `clear_caches()` makes a cold run explicit, and `cache_info()` exposes hits and misses.

These are bounds on *cache entry counts*, not a bound on total bytes. A single morphism can have exponentially many monomials. The underlying low-level topology records are internal shared objects and must be treated as read-only; the tests specifically check nonmutation of returned public morphisms. No polynomial-space theorem follows merely from adding an LRU limit.

5.3 Cancellation and fill-in control

If a differential component $\phi : b \rightarrow c$ is invertible, Gaussian cancellation removes b, c . For any remaining incoming component $\delta : a \rightarrow c$ and outgoing component $\gamma : b \rightarrow f$, replace the surviving differential by

$$d'_{af} = d_{af} + \gamma\phi^{-1}\delta. \quad (7)$$

The sign is plus over $\mathbb{F}_2$. This is the Schur complement form of Bar–Natan’s Gaussian elimination lemma and preserves chain homotopy type [4]. Algebraically, triangular basis changes isolate the isomorphism ϕ as a contractible two-object summand; deleting that summand gives (7). The adjacent differentials transform consistently because $d^2 = 0$.

Correctness does not prescribe the order of unit cancellations. Performance can change sharply because different orders create different intermediate entries. For a candidate $b \rightarrow c$, define

$$q(b, c) = (|\text{inc}(c)| - 1)(|\text{out}(b)| - 1).$$

It estimates the number of possible Schur corrections, not their actual nonzero count. A heap prioritizes small q , breaking ties by the sum of the two degrees and then object IDs. Removed or nonunit candidates are discarded. An underestimated stale priority is refreshed and reinserted; a stale *overestimate* may be accepted. Thus `minfill` is a lazy fill-in-aware heuristic, *not* a guarantee of the globally cheapest pivot at every step. Every accepted cancellation is nevertheless exact.

The old last-in-first-out choice remains available as `pivot_strategy="stack"` in the raw Python API. When there are no outgoing correction targets, the implementation skips even the unnecessary incoming half-compositions. Otherwise $\phi^{-1}\delta$ is computed once per incoming entry and reused. Composition statistics in this version count these actual requests; the old counter charged some half-compositions differently. Timings are compared across versions, but the old and new raw composition counters are not presented as identical performance units.

5.4 Faster crossing orders

The baseline’s greedy order repeatedly examines all remaining crossings. The priority first maximizes the number of edges shared with the current frontier, then prefers self-loops, then the lowest crossing index. For an unprocessed crossing, the shared-edge count changes only when one of its neighbors is processed. It never decreases before that crossing is selected. The replacement builds adjacency counts once and maintains a lazy heap of these same priorities.

Proposition 5.2 (Order construction). *For any validated n -crossing PD and specified starting crossing, the new routine returns exactly the baseline greedy order using $O(n \log(n + 2))$ word operations and $O(n)$ words of storage.*

Proof. There are $2n$ graph edges, counted with multiplicity. Each edge between different crossings causes at most one priority update while its second endpoint remains unprocessed. Initial heap entries and all updates therefore number $O(n)$. Each heap operation costs $O(\log(n + 2))$. An obsolete priority is ignored, and the current priority of every eligible crossing is present. Lexicographic comparison implements exactly the baseline tie rules. Loops contribute only the fixed tie score and cause no neighbor update. $\square$

Trying r sampled starts costs $O(rn \log(n + 2))$ instead of $O(rn^2)$. The inherited start-sampling expression can use somewhat more than the requested number of starts, but still $O(r)$; the default is a fixed number independent of n . Asking for *all* starts gives $O(n^2 \log n)$, not the fixed-start bound. The resulting order is heuristic: no optimal-cutwidth claim is made.

6 Preprocessing, validation, and evidence

6.1 A linear descending test

A knot traversal has $2n$ crossing visits. Relative to a chosen start, mark a crossing bad when its first visit is under. Moving the start past one visit changes the first-visit choice of exactly that crossing; all other first-visit choices retain their order. Passing an over-visit increments the bad count, and passing an under-visit decrements it. A zero count gives a descending start. The routine performs an initial pass and a sliding pass in each traversal direction, returning the least valid dart just as the baseline does. This gives $O(n)$ word operations instead of testing $O(n)$ starts independently. A descending diagram represents the unknot: lifting successive first encounters above later ones permits the traversed arc to be straightened without changing any crossing.

6.2 An orientation fix needed for safe normalization

The old sign routine looked only at the incoming over-port. A PD crossing may be rotated by 180 degrees without changing it, and that operation can change the absolute incoming port numbers. The corrected routine uses the relative position of the incoming over-port o and incoming under-port u :

$$\text{sign} = +1 \iff (o - u) \bmod 4 = 3.$$

The relative difference is invariant under simultaneously rotating the ports by two, and under reversing the strand orientation. The Alexander matrix was updated at the same time to use the actual incoming and outgoing under-arcs; changing only the sign routine would not be a safe patch. The bracket normalization, mirroring, local port rotations, and Reidemeister tests use the same convention throughout.

6.3 Limits are not mathematical answers

The raw scanning API now validates the PD and requires an explicit scan order to be a permutation of *all* crossing indices. Incomplete, repeated, negative, out-of-range, and Boolean indices are rejected. Time limits must be finite and nonnegative, and object and state ceilings must be positive integers when set. The global deadline is shared across factor analyses and preprocessing. Additional checks occur during scanning, simplification, and the Alexander Bareiss calculation. New-object ceilings are tested while preparing the tensor expansion, rather than only after allocating every new object.

Time limits are cooperative, not hard real-time or operating-system memory limits. One sizeable elementary operation can overrun a deadline before the next check. The object ceiling bounds active object creation, not morphism bytes, caches, or all process memory. The benchmark harness adds an external subprocess timeout for isolation. Under these limits, failure to complete returns UNKNOWN; it must never be interpreted as KNOTTED.

6.4 Replayable evidence and its trust boundary

Recognition outputs contain the Reidemeister trace, any descending start, decomposition tree and factor results, modular evaluation data, Alexander coefficients, or Khovanov ranks and scan order as appropriate. The `verify_result` routine replays local moves and dual cuts; recomputes Jones/Alexander obstructions; and, for a Khovanov verdict, recomputes the scan with $d^2 = 0$ checks after every crossing. A knotted connected sum needs a verified nontrivial factor; an unknot sum needs every factor verified as an unknot.

This is reproducible mathematical evidence, not a claim of a succinct polynomial-size certificate for arbitrary Khovanov computations. The verifier shares diagram and algebra routines with the recognizer. Its Khovanov branch may be as expensive as computing the invariant again. Neither Python nor the cobordism implementation has been formalized in Lean. Independent reference algorithms and invariance tests reduce common-mode risk, but do not eliminate that trust boundary.

7 Complexity: achieved improvements and remaining barriers

7.1 General worst case

For an n -crossing connected diagram, there are 2^n resolutions and at most $n + 1$ circles in a resolution, so the unreduced full cube has at most 2^{2n+1} generators. The analogous partial scanning objects, their dot-normal forms, and differential entries have elementary exponential upper bounds as well. Gaussian elimination and composition use a fixed number of polynomial loops in these exponentially bounded quantities. Consequently an unbounded execution of the supplied style of algorithm has the conservative bound $2^{O(n)}$, including polynomial-size bit bookkeeping. The bounded Jones screen and the polynomial diagram operations do not worsen this bound.

Neither the algebraic shortcuts nor the pivot heuristic changes that global worst-case classification. In particular, no empirical regression on a few timing points is used to infer an asymptotic exponent.

7.2 Why width alone does not bound a materialized Khovanov complex

Theorem 7.1 (A fixed-width multiplicity obstruction). *There is a family of knot diagrams with $n = O(k)$ crossings and scan orders of bounded frontier width for which the final reduced Khovanov rank is r^k with $r > 1$. Any scanner that represents each final homology generator by a separate object needs exponentially many final objects on this family.*

Proof. Fix a diagram of a nontrivial knot K . Join k copies in a visible linear connected sum. The copies have bounded size and can be scanned one at a time; only a bounded number of connection edges and one fixed-size block are active. Thus there is a constant-width order, independent of k . By the connected-sum formula, the reduced rank is $r(K)^k$. By (5), $r(K) > 1$. An explicit final basis requires at least that many objects, independent of how unit eliminations were scheduled earlier. $\square$

The integer r^k itself has only $O(k)$ bits. Thus this is a barrier to the *representation used by the supplied scanner*, not an output-size lower bound for computing a rank in binary, and not a lower bound for all unknot recognition algorithms. It explains why ordinary memoization of repeated matching *types* does not suffice: many distinct surviving generators can have exactly the same matching type.

This also corrects a separate width statement in the supplied README. An n -vertex planar graph of maximum degree four admits a vertex scan order with $O(\sqrt{n})$ crossing edges, rather than having optimal width $\Theta(n)$. Indeed, recursively take balanced planar separators of $O(\sqrt{n})$ vertices [9], process a separator and then its remaining components, and sum the at-most-four incident edges per separator vertex along the active recursion chain:

$$O(\sqrt{n}) \sum_{j \geq 0} (\sqrt{2/3})^j = O(\sqrt{n}).$$

The implementation’s greedy ordering need not find such an order, and we do not implement a separator-order algorithm in this release. More importantly, even *constant* width does not remove the generator-multiplicity obstruction. A boundary-only complexity assertion for the raw Khovanov representation would therefore remain false after improving the order.

7.3 An exponential improvement for the actual recognition pipeline

Theorem 7.2 (Structured-family speedup). *Fix a nontrivial Alexander-trivial knot K and a fixed diagram for it. On the family of visibly spliced copies $K^{\#k}$, the supplied baseline recognition pipeline, if run to completion, uses $\Omega(r(K)^k)$ final Khovanov objects. The improved recognizer and its factored rank routine run in polynomial time in k when the validated decomposition extracts bounded-size blocks.*

Proof. The represented knot is nontrivial by the rank product, so sound Reidemeister and descending tests cannot certify it as an unknot. Its Alexander polynomial is one, because Alexander polynomials multiply under connected sum. The baseline therefore reaches its Khovanov routine. Simplifying the diagram does not change the final rank $r(K)^k$, so that routine must materialize exponentially many objects.

In the improved pipeline, finding all validated visible cuts costs polynomial time. Each extracted fixed-size block can be analyzed in constant time with respect to k , even if it requires its complete Khovanov computation. There are $O(k)$ blocks. Recognition may stop at the first certified nontrivial one. For the factored rank routine, the remaining integer coefficient convolutions have polynomial bit complexity. No tensor-product basis is expanded. $\square$

The Conway fixture realizes the intended difficult-filter case: its Alexander polynomial is one, while independent cube and scanning computations both give $r = 33$. The test family is generated by the included `connected_sum` helper, and all decomposition cuts are replayed. This theorem is more than a constant-factor micro-optimization, but its hypothesis is a visible decomposition, not arbitrary topological compositeness.

Corollary 7.3 (A precise restricted quasipolynomial bound). *Let b be the largest crossing count of any block actually returned by the validated diagram decomposition. The improved complete recognizer has the conservative bound*

$$T(n, b) \leq \text{poly}(n) + n 2^{O(b)}.$$

In particular, bounded b gives polynomial time; $b = O(\log n)$ also gives polynomial time; and $b = O(\log^2 n)$ gives $n^{O(\log n)}$.

Proof. The factor crossing counts sum to n , hence there are at most n nonempty blocks. Each block’s complete analysis has exponential upper bound in its own crossing count, at most $2^{O(b)}$, plus polynomial preprocessing. Decomposition and bookkeeping contribute the polynomial term. Substitution gives the three specializations. $\square$

When no visible cut exists, $b = n$ and this is only the ordinary exponential bound. The corollary does not establish the requested global quasipolynomial bound, nor does it say that every composite knot has a conveniently factored projection.

8 Reproducible measurements

8.1 Protocol and environment

All numbers in Tables 1–4 come from the included `benchmarks/comparison.json` and are regenerated into LaTeX by `tools/make_report_tables.py`. The measured environment was CPython 3.13.5, GCC 14.2.0, Linux x86_64 with glibc 2.41. The processor reported **AMD EPYC 9V74 80-Core Processor**; the container reported five logical CPUs. Timings are single-process measurements in a shared container, not isolated-hardware performance guarantees.

Each sample runs in a fresh child process, so caches start cold. The reported timer excludes imports, process startup, JSON parsing, and initial **Diagram** construction. Thus these are algorithm timings on an already constructed diagram, not command-line wall times. The improved raw scan performs its own internal revalidation, which *is* counted; this gives it a small extra safety cost absent from the old raw function. Every successful group has three samples; the tables use the median. Raw minimum and maximum times and all full result records are retained. A censored group is attempted once, with a 30-second cooperative limit and an outer subprocess timeout three seconds later.

The comparison is against the unchanged supplied source, not a recreated approximation. Small-input timings are especially sensitive to runtime noise. The last source hardening added only argument rejection and clarified comments; valid benchmark paths are unchanged. The final 48-test suite was then rerun.

8.2 Default recognition

Input	Baseline (ms)	Improved (ms)	Ratio
Conway (11 crossings)	35.090	0.834	42.09×
Kinoshita–Terasaka (11)	37.594	0.904	41.58×
Hard unknot (8)	1.622	2.244	0.72×
Trefoil (3)	0.092	0.229	0.40×
Figure-eight (4)	0.125	0.272	0.46×
Torus $T(3, 5)$ (10)	0.588	0.860	0.68×
Grid-scrambled unknot	0.320	0.280	1.14×
Unknot braid (40)	6.448	6.264	1.03×
Four-strand braid (41)	10.679	5.908	1.81×
Five-strand braid (36)	8.626	5.043	1.71×
Conway #2 (22)	1,372.423	1.455	943.04×
Conway #3 (33)	> 30,000	1.972	> 15,210.39×

Table 1: Default recognition, cold-process medians. Ratio is baseline time divided by improved time; ratios below one are regressions. The $>$ marker denotes a censored baseline, not a measured completion time.

The largest improvement among completed default comparisons is the two-Conway sum, about 943×. The three-Conway baseline does not complete within the 30-second limit, while the new default completes in about 1.97 ms. These inputs are Alexander-trivial, which is precisely why the old pipeline computes the large Khovanov complex. The single Conway and Kinoshita–Terasaka fixtures are rejected quickly by the Jones screen without computing homology.

The result is not uniformly faster. The trefoil, figure-eight, and $T(3, 5)$ fixtures were already rejected cheaply by Alexander; the added screen and cut checks introduce overhead. The hard eight-crossing unknot is another regression in default recognition: an agreeing Jones evaluation

is correctly inconclusive and the complete Khovanov fallback still has to run. Flags to disable Jones and factorization permit the user to retain the old filter order when a workload consists mainly of such easy cases.

8.3 Raw scanning, separated from preprocessing

Input	Baseline (s)	Improved (s)	Ratio
Conway (11 crossings)	0.034	0.015	2.30×
Kinoshita–Terasaka (11)	0.045	0.016	2.73×
Hard unknot (8)	0.002	0.002	1.11×
Torus $T(3, 5)$ (10)	0.010	0.010	1.02×
Four-strand braid (41)	0.923	0.283	3.26×
Five-strand braid (36)	> 30	1.487	> 20.18×
Conway #2 (22)	1.395	0.235	5.93×

Table 2: Raw Khovanov scans: no Reidemeister reduction, no Jones/Alexander filter, and no connected-sum factorization. Both completed versions return the same reduced ranks.

This isolates improvement of the supplied backend itself. On the four-strand 41-crossing stress braid, the median drops from 0.923 s to 0.283 s; both return reduced rank 109. On the two-Conway sum, the raw median drops from 1.395 s to 0.235 s. Both versions still materialize 2178 final unreduced objects on that case, with 8118 objects at the pre-elimination peak. The gain here is faster algebra, ordering, and cancellation management, not the compressed sum algorithm.

For the five-strand 36-crossing fixture, the new raw scan returns reduced rank 2949 and unreduced rank 5898 in 1.487 s median, with 17694 objects at the pre-elimination peak and boundary width eight. The original raw scan does not finish within our 30-second limit, so the comparison supports a conservative speedup greater than 20×, not an exact ratio.

The archive’s old *historical* benchmark recorded a 600.147-second timeout for this same raw case under Python 3.14.4 on Windows 11. That is useful for identifying the stress input but is not a same-machine baseline. We do not divide 600 seconds by a Linux timing to advertise a speedup. Also, *default* recognition of the same 36-crossing input is already fast in the baseline: 8.626 ms versus 5.043 ms here. The dramatic raw-scan improvement must not be confused with default end-to-end recognition on that input.

8.4 Compressed rank computations

Input	Crossings	Reduced rank	Time (ms)	Peak factor objects
Conway #2	22	33^2	24.783	246
Conway #3	33	33^3	25.451	246
Conway #8	88	33^8	30.007	246
Conway #16	176	33^{16}	47.949	246
Trefoil #8	24	3^8	3.374	18
Trefoil #16	48	3^{16}	7.490	18

Table 3: Complete reduced-rank and homological-degree computations using the validated factorization. “Peak factor objects” is the largest pre-elimination object count of any individual factor scan, not total process memory.

For 16 Conway copies, the program returns

$$33^{16} = 1\,977\,985\,201\,462\,558\,877\,934\,081$$

as reduced rank, together with the full raw homological-degree rank map, in about 47.95 ms median. Only two distinct normalized factor PDs are computed in this fixture; the other 14 reuse an identical tuple. The maximal individual factor scan has 246 objects before elimination. This does not mean the whole process uses 246 objects' worth of memory: the decomposition records, cached algebra, factor results, and integer polynomial maps are also stored.

On two Conway copies, the factored complete rank takes 24.78 ms, as compared with 235.42 ms for the improved raw scan and 1395.42 ms for the old raw scan. This is distinct from the much cheaper recognition-only time, which stops after one nontrivial factor is found.

8.5 Ordering and pivot ablations

n	Baseline (ms)	Heap order (ms)	Ratio
128	55.575	3.078	18.06×
256	234.410	5.771	40.62×
512	890.923	12.638	70.50×
1024	3,742.642	81.868	45.72×

Table 4: Order construction alone on the closure of $\sigma_1 \cdots \sigma_n$. Both algorithms return the same chosen order. These timings include all sampled greedy starts, but not a homology scan.

The order-only experiment isolates the asymptotic improvement from $O(n^2)$ to $O(n \log n)$ per start. On the $n = 512$ fixture, the medians are 891 ms and 12.64 ms. At $n = 1024$ the improved median is 81.87 ms; the observed curve is not asserted to be a precise asymptotic fit. Both versions return the same orders in the differential order tests. On the 256-crossing unknot chain, the entire raw scan decreases from 250.04 ms to 22.80 ms.

Input	Cached algebra, stack (s)	Cached algebra, heap (s)
Conway (11 crossings)	0.014763	0.013504
Kinoshita–Terasaka (11)	0.015803	0.015057
Five-strand braid (36)	> 10	1.497090

Table 5: Pivot ablation: both columns use the new cached algebra and order routine. Fresh processes, median of three successful samples, ten-second limit for the censored stack case.

Table 5 is stored separately in `benchmarks/ablation.json`. The small named examples gain only modestly from changing stack to heap pivots once the algebra is already accelerated. The 36-crossing case remains incomplete at ten seconds with stack pivots, while heap pivots complete in 1.497 s median. This supports the practical importance of pivot order on that case; it is neither a general speed guarantee nor a proof that the heuristic minimizes fill-in.

9 Validation actually performed

The final command

```
PYTHONPATH=fast python -m unittest discover -s tests -v
```

completed with **48 tests passing in 8.538 seconds**; the full log is in `benchmarks/tests.log`. These are test methods, several of which contain many independently generated cases. The unchanged baseline's 19 tests were also run separately and passed. The new test suite retains those 19 legacy methods, explicitly disabling the new filters where an old method choice is being asserted, plus 29 additional methods.

The algebra tests compare every one of the 128 units in a three-arc endomorphism algebra against the old inverse routine. They compare 1200 random valid planar six-endpoint cobordism compositions against the unchanged composition routine, and check that mutating a public result does not mutate its cached value. Sixty random scans compare both pivot strategies and the old backend, with differential-square checks. An independent dense reduced cube-of-resolutions implementation is tested on 80 random small braid diagrams and on the Conway, Kinoshita–Terasaka, hard-unknot, and $T(3, 5)$ fixtures.

The Jones tests evaluate 100 random diagrams by an independent full state sum on $4n$ crossing darts, at three choices of modular parameters. That reference does not call the frontier transition or cobordism gluing code. Additional checks cover small unknots, diagram relabeling and crossing reordering, 180-degree port rotations, mirrors, Reidemeister-II insertion and the braid relation, nonprime moduli, and $\delta = 0$. Equality is explicitly tested to remain inconclusive.

Connected-sum tests compare raw and factored ranks and their degree convolutions, mixed summands, repeated Conway factors, and sums of unknots. Malformed cuts, duplicate tree data, and tampered evidence are rejected. The linear descending test matches the baseline on 300 random diagrams; greedy orders match on 90 random diagrams at every possible starting crossing. Invalid orders, parameter values, ceiling exhaustion, skipped Jones screens, CLI exit codes, and result replay are tested too.

The 36-crossing stress scan is tested with $d^2 = 0$ after every stage. A separate script, `tools/validate_stress.py`, compares it with the diagram obtained by six valid R2 moves, reducing 36 crossings to 24. Both complete scans return reduced rank 2949, and their full raw homological-degree maps agree after the expected shift by six. The reduced diagram's Alexander coefficient list is

$$(2, -19, 80, -206, 372, -512, 567, -512, 372, -206, 80, -19, 2),$$

whose absolute evaluation at -1 is also 2949. This determinant agreement is a sanity check, *not* a general theorem equating determinant and Khovanov rank. The large-diagram invariance check uses the same scan backend on two Reidemeister-related inputs, so it is not an independent large Khovanov calculation. Independent dense-cube checks are limited to feasible small cases.

All completed baseline/improved benchmark pairs have consistent mathematical outputs. Four included example recognition records have been replayed: Conway, the hard eight-crossing unknot, three Conway copies, and the 36-crossing braid. Their evidence files are in `examples/evidence`. These experiments support correctness and reproduce the reported improvements; they do not prove that no implementation bug remains on arbitrary inputs.

10 Using and extending the implementation

From the unpacked archive root, these commands need no installation:

```
cd fast
python -m fastunknot recognize examples/conway.json
python -m fastunknot recognize examples/conway.json --output result.json
python -m fastunknot verify examples/conway.json result.json
python -m fastunknot jones examples/conway.json
```

```
python -m fastunknot khovanov examples/conway.json
python -m fastunknot khovanov examples/conway.json --no-factor
```

The last two commands distinguish compressed-factor homology from a raw scan. To use the original pipeline stages with the improved backend:

```
python -m fastunknot recognize examples/conway.json \
    --no-jones --no-factor
```

Optional flags include `--seconds`, `--max-objects`, `--jones-max-states`, `--check-d2`, and switches disabling individual preprocessors. Exit code 0 means a completed mathematical result; code 2 means invalid input or rejected evidence; code 3 means resource-limited UNKNOWN. A standalone `jones` command returns an evaluation, not an unknot verdict.

For an importable installation, Python 3.10 or later is required:

```
python -m pip install ./fast
```

There are no runtime dependencies beyond the standard library. Example Python use:

```
from fastunknot import Diagram, recognize, verify_result
from fastunknot import khovanov_rank, factored_khovanov_rank

D = Diagram.from_braid(2, [1, 1, 1])
result = recognize(D, seconds=10)
print(result.status, result.method)
if result.status != "UNKNOWN":
    assert verify_result(D, result.to_json(), seconds=10)

raw = khovanov_rank(D.pd, pivot_strategy="minfill")
compressed = factored_khovanov_rank(D)
assert raw["reduced_rank"] == compressed["reduced_rank"]
```

The CLI keeps the existing input formats. For example,

```
{"braid": {"strands": 2, "word": [1, 1, 1]}}
```

is a three-crossing braid closure. Arbitrary integer PD edge labels are normalized by `Diagram.from_pd`. Direct construction of the low-level `Diagram` dataclass bypasses those checks and is not the supported way to validate untrusted input.

To reproduce the experiments and rebuild this report, return to the archive root:

```
python tools/make_inputs.py
PYTHONPATH=fast python -m unittest discover -s tests -v
python tools/benchmark.py --repeats 3 --timeout 30
python tools/ablate.py --repeats 3 --timeout 10
python tools/validate_stress.py
python tools/make_report_tables.py
cd report
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The first command regenerates the structured connected-sum and chain fixtures. The original random-braid stress fixtures are already supplied and are preserved verbatim in meaning; their source words and provenance are recorded. A quick benchmark subset is available with `--quick`.

On Windows, installation via `pip` avoids shell-specific `PYTHONPATH` syntax. The test files also set the import path themselves.

Useful further engineering work would include better exact workload scheduling between Jones and Alexander, a disk-aware separator decomposition for the Jones state table, and more general compression of surviving Khovanov multiplicities. A tuple cache alone cannot provide the latter: interactions between equal matching types may require nontrivial differential structure. For the hierarchy route, the next deliverable should be independently checked compressed cutting and boundary-pattern operations with explicit bit-size contracts, not a renamed call to the exponential fallback.

11 Conclusion

The delivered recognizer preserves exact decision semantics and improves both its early obstruction stages and its complete homological backend. The characteristic-two inverse shortcut and lazy pivot strategy speed up genuinely raw scans. Validated sum decomposition eliminates an exponential representation cost on a concrete infinite family, including Alexander-trivial examples that force the original recognition pipeline into Khovanov homology. The factored rank routine demonstrates that a huge rank need not imply a huge explicit basis.

The general $n^{O(\log n)}$ target remains unproved for this implementation. The supplied Lackenby paper supplies real compressed-topology ingredients, but turning the talk’s logarithmic-depth strategy into an executable with complete size and time accounting is a different construction. The accompanying source, measurements, proofs, and test logs distinguish the improvements achieved here from that remaining objective.

References

- [1] *Knots.zip*, archive supplied with the task, specifically `fast/fastunknot`, `fast/tests`, `fast/results/benchmark.json`, and `docs`. The original Python sources and relevant benchmark provenance are retained in this deliverable. Accessed 18 September 2026.
- [2] Marc Lackenby, *Unknot recognition in quasi-polynomial time*, February 2021. Supplied `docs/quasipolynomial-talk.pdf`; 109 physical pages. In particular, physical slides 14, 66–74, and 109.
- [3] Marc Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 2026. Supplied source `docs/arXiv-2607.23350v1/algorithm-incompressible-250726.tex`.
<https://arxiv.org/abs/2607.23350>.
- [4] Dror Bar-Natan, *Fast Khovanov homology computations*, Journal of Knot Theory and Its Ramifications **16** (2007), 243–255. arXiv:math/0606318.
<https://arxiv.org/abs/math/0606318>.
- [5] Dror Bar-Natan, *Khovanov’s homology for tangles and cobordisms*, Geometry & Topology **9** (2005), 1465–1499.
<https://www.math.utoronto.ca/drorbn/papers/Cobordism/Cobordism.pdf>.
- [6] Peter B. Kronheimer and Tomasz S. Mrowka, *Khovanov homology is an unknot-detector*, Publications Mathématiques de l’IHÉS **113** (2011), 97–208. arXiv:1005.4346.
<https://arxiv.org/abs/1005.4346>.
- [7] Alexander N. Shumakovitch, *Torsion of the Khovanov homology*, Fundamenta Mathematicae **225** (2014), 343–364; arXiv:math/0405474, v2 (2018), especially Corollary 3.2.C. <https://arxiv.org/abs/math/0405474>.
- [8] Louis H. Kauffman, *State models and the Jones polynomial*, Topology **26** (1987), no. 3, 395–407. [https://doi.org/10.1016/0040-9383\(87\)90009-7](https://doi.org/10.1016/0040-9383(87)90009-7).
- [9] Richard J. Lipton and Robert E. Tarjan, *A separator theorem for planar graphs*, SIAM Journal on Applied Mathematics **36** (1979), no. 2, 177–189.
<https://www.jstor.org/stable/2100927>.